Dinamik Veri Yapısı

* Dinamik veri yapısı, programın çalışma zamanında ihtiyaca doğrultusunda büyüyebilen ve küçülebilen veri yapısıdır.

Static Veri Yapısı

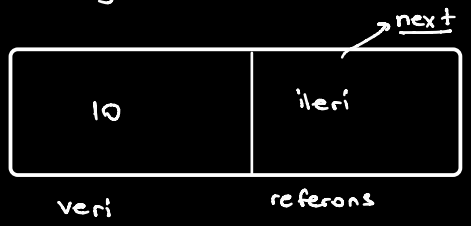
* Statik veri yapısı, boyutu genellikle oluşturulurken belirlenen çalışma sırasında doğrudan kaçulup büyüyememeyen veri yapısıdır.

Özellik	Static	Dinamik
Boyut	Genellikle sabit	Değişebilir.
Boyut Belirleme	Önceden	Çalışma zamanında
Bellek	Önceden ayrılır	İhtiyaca göre
Esneklik	Düşük	Yüksek
Yönetim	Kolay	Karmaşık
Ek bellek Maliyeti	Düşük	Bağlantılar vb. olabilir
Örnek	Array	Linked list
Bellek Yönetimi	Basit	Dikkat gerekir.

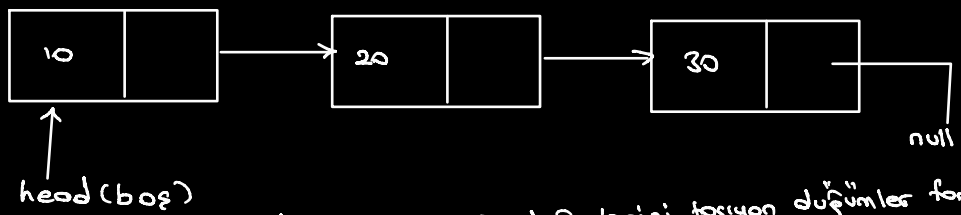
II. Bölüm - Linked List

Linked List

* Linked list, elemanların art arda bulunmak zorunda olmadığı, birbirine referans bağlar diği dinamik veri yapısıdır.
* Bir bağlı listenin her bir elemanına node(eleman, düğüm) denir.
* Tek yönlü bağlı liste bir düğüm iki temel şeyi taşır.
  - veri: Düğümde saklanan bilgi
  - ileri: Bir sonraki düğümün referansı
* Tek bir düğüm yapısı;



* Birbirine bağlı üç düğüm:



- Bolek mantığı açısından 10, 20 ve 30 değerlerini taşıyan düğümler farklı adreslerde bulunabilir. Bağlı ilişkiyi oluşturan şey, bu düğümlerin adresi değil, ileri referansıyla kurduğu zincirdir.

Java ile Linked List oluşturma

* Derste hocanın yazdığı kodlara göre oluşturulmuştur.

Eleman.java ⇒ Linked listeki veri ve ileri oluşturur.

```
public class Eleman {
    int veri;
    Eleman ileri;          → yerel değişken

    Eleman (int veri) {    } Constructer (yapılandırıcı)
      this.veri = veri;
      this.ileri = null;
    }
}
```

BağlıListe.java ⇒ Temel iskelet

```
public class BagliListe {
    Eleman bas;
    BagliListe() {
      bas = null;
    }
}
```

* bas == null bağlı listenin boş olduğunu gösterir.
* İlk eleman eklendiğinde boş olduğunu gösterecek. Bağlı listeye boşlar. Diğer düğümlere ulaşmak için başı kullanır.

2

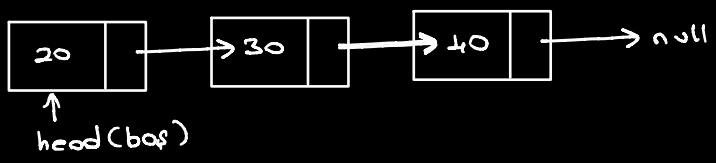
Linked liste Ekleme

* Bağlı listede ekleme genel olarak 4 şekilde incelenir:

1. Başa ekleme
2. Sona ekleme
3. Belirtilen konuma ekleme
4. Belirlen değerin sonuna veya öncesine ekleme

1. Başa ekleme (Insert at Beginning)

- Başlangıç listesi:



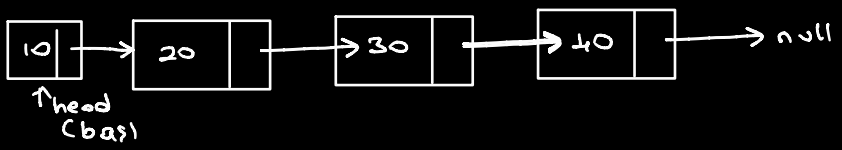
- Eklenecek değer



Algoritma Mantığı

"Önceden yeni düğüm eski başa bağlanır ardından baş yeni düğümü gösterir.

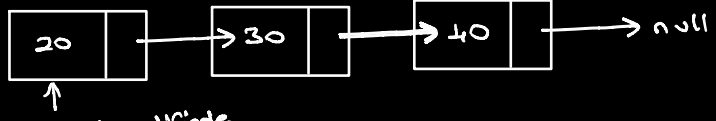
- 10 başa eklendiğinde




```
void basaEkle(int veri){
    Eleman yeni = new Eleman(veri);
     yeni.ileri = bos;
     bos = yeni;
}
```

2. Sona ekleme (Insert at End)

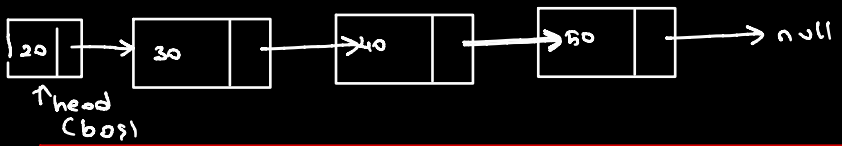
- Başlangıç listesi:



- Eklenecek değer



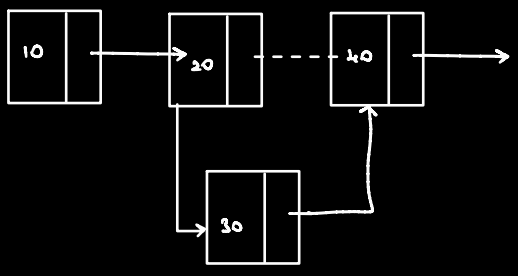
- 50 sona eklendiğinde




```
void sonaEkle(int veri){
    Eleman yeni = new Eleman(veri);
    Eleman gezici = bos;
    if(bos == null){
        bos = yeni;
        return;
    }
   while(gezici.ileri != null){
        gezici = gezici.ileri;
    }
    gezici.ileri = yeni;
}
```

3. Belirli bir konuma ekleme (Insert at index)

- Örneğin, indexler 0 başlıyor ve 10→20→40 listesine 2. konumuna 30 eklenirse



```
void konumunaEkle(int index, int veri){
  Eleman gezici = bos
  if(index < 0){
    System.out.println("index 0'dan kucuk olmaz");
  }
  if(index == 0){
     basaEkle(veri);
     return;
  }
  for(int i=0; i < index-1; i++){
      if(gezici == null){
        system.out.println("Listede eleman yoktur.");
      }
      gezici = gezici.ileri;
  }
  if(gezici == null){
    system.out.println("Listede eleman yoktur.");
  }
  Eleman yeni = new Eleman(veri);
  yeni.ileri = gezici.ileri
  gezici.ileri = yeni;
}
```

4. Belirli bir değerin arkasına ekleme

- "Örneğin 10 → 20 → 30 listesinde 20 sonrasına 30 ekleme

```
boolean ArkasinaEkle(int hedef, int veri){
      Eleman gezici = bos;
      while(gezici.ileri != null){
          if(gezici.veri == hedef){
             Eleman yeni = new Eleman(veri);
             yeni.ileri = gezici.ileri;
             gezici.ileri = yeni;
            }
          gezici = gezici.ileri;
       }
    return false;
  }
```

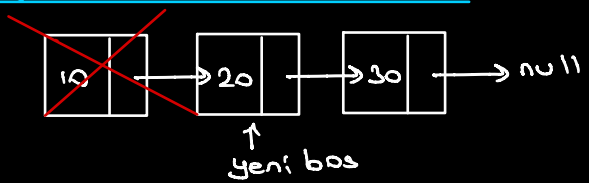
5. Belirli Bir değerin önüne eklemek

```
boolean OnuneEkle(int hedef, int veri){
    if(bos == null) return false;
    if(bos.veri == hedef){
       bosEkle(veri);
       return true;
    }
    Eleman onceki = bos;
    Eleman gezici = bos.ileri;
    while(gezici != null){
       if(gezici.ileri == hedef){
          Eleman yeni = new Eleman(veri);
          yeni.ileri = gezici;
          onceki.ileri = yeni;
          return true;
       }
       onceki = gezici;
       gezici = gezici.ileri
    }
  return false
}
```

Linked List Silme Türleri (Deletion)

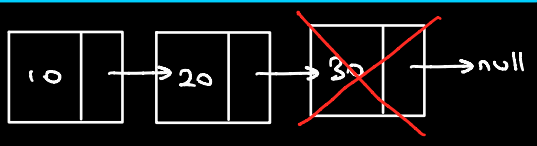
* Eklemede düğümleri bağlarız, silmede zincirden çıkarılır.
* Java'da bir düğüm artık erişilebilir değilse, belleğe uygun zamanda Garbage Collector tarafından geri kazanılabilir.

1. Baştan silme (Delete First)



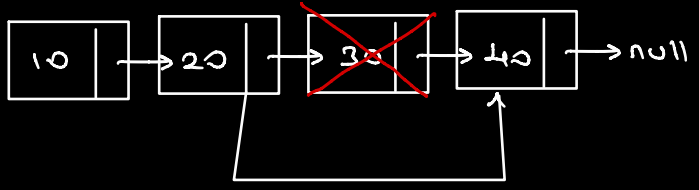

```
void bastanSil(){
    if(bos == null){
        System.out.println("Liste bos.");
        return;
    }
    bos = bos.ileri;
}
```

2. Sondan Silme işlemi (Delete Last)




```
void sondanSil(){
    if(bos == null){
        System.out.println("Liste bos.");
        return;
    }
    Eleman gezici = bos;
    while(gezici.ileri.ileri != null){
        gezici = gezici.ileri;
    }
    gezici.ileri = null;
}
```

3. Belirli bir değer silme (Delete By Value)




```
boolean degerSil(int hedef){
    if(bos == null) return false;
    if(bos.veri == hedef){
        bos = bos.ileri;
        return false;
    }
    Eleman gezici = bos;
    while(gezici.ileri != null){
        if(gezici.ileri.ileri == hedef){
            gezici.ileri = gezici.ileri.ileri;
            return true;
        }
        gezici = gezici.ileri;
    }
    return false;
}
```

4. Bütün Listeyi Silmek

```
void listeyiTemizle(){
    bos = null;
}
```

9. Belirlenen indexe göre silme

```
void konumdansil(int index, int veri){
    if(index <0 || bas ==null){
      throm new IndexOutOf Arguments;
    }
    if( index ==0){
      bas= bas.ileri
      return;
    }
    Elemon gecici =bas;
    for(int i =0; i<index-1;i++){
        if( gecici -=null){
          throw new IndexOutArguments;
          }
          gecici =gecici.ileri;
        }
      if (gecici == null || gecici.ileri ==null){
        throw new IndexBoundsExeption();
      }
      Gecici.ileri =gecici.ileri.ileri;
  }
```

Linked Liste Aroma Türleri

1. Doğrusal Aroma (Linear Search)

* Tek yönlü bağlı listede ilk düğümden başlanır ve hedef bulunana kadar devam edilir.

```
boolean ara(int hedef) {
    Elemon gezici = bas;
    while( gezici != null ){
        if(gezici.veri == hedef){
            return true;
        }
        gezici = gezici.ileri;
    }
    return false;
}
```

2. Index döndüren Aroma

```
int indexBul (int index, int hedef){
    Elemon gezici=bas;
    int index =0;
    while (gezici != null ){
        if(gezici.veri == hedef){
            return index;
        }
        index++;
        gezici = gezici.ileri;
    }
    return -1;
}
```

3. Bütün konumlara göre Arama

```
void tumKonumlariBul(int hedef){
    Eleman gezici = bas;
    int index = 0;
    while(gezici!=null){
      if(gezici.veri==hedef){
        System.out.println("Bulundu: "+index);
      }
      gezici=gezici.ileri;
      index++;
    }
  }
```

4. Recursive Arama

```
boolean recursiveAra(Eleman dugum, int hedef){
    if(dugum==null) return false;
    if(dugum.veri == hedef) return true;
    return recursiveAra(dugum.ileri, hedef);
  }
```

Linked Liste Gezinme

* Gezinme, bir bağlı listedeki düğümleri sırayla ziyaret etme işlemidir.
* Arama belirli bir değeri bulmayı amaçlarken gezinme bütün değerleri işlemek için kullanılır. Örneğin elemanları ekrana yazdırmak toplam hesaplamak gibi durum lerde kullanılır.

1. while döngüsüyle gezinme

```
void listele(){
  Eleman gezici=bas;
  while(gezici!=null){
    System.out.println(gezici.veri + "->");
    gezici = gezici.ileri;
  }
  System.out.println("null");
}
```

2. for döngüsü ile gezinme

```
void forIolisteleme(){
    for(Eleman p=bas; p!=null; p=p.ileri){
      System.out.println(p.veri);
    }
```

3. Recursive gezinme

```
void recursiveListele(Eleman dugum){
    if(dugum==null) return;
    System.out.println(dugum.veri);
    recursiveListele(dugum.ileri);
}
```

Linked Liste Sorgulama

*Sorgulama işlemleri listenin mevcut durumu hakkında bilgi üretir. Genellikle liste yapısını değiştirmez.

1. Liste boş mu sorgusu:

```
boolean basmu(){
    return bas == null;
}
```

2. Elemon Sayısı Sorgusu:

```
int elemonSay(){
  int Soyac=0;
  Elemon gezici = bas
  while (gezici != null){
      Soyac ++;
      gezici = gezici.ileri;
    }
  return soyac;
}
```

3. Orta Düğümü bulma

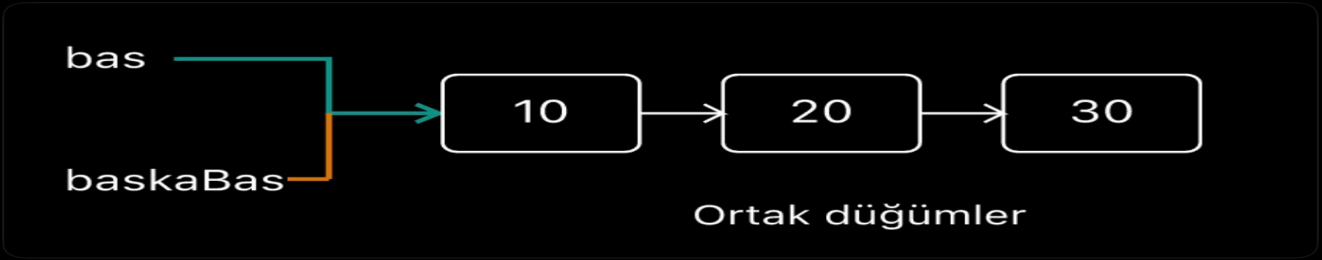
```
int ortaElemon(){
   if( bas == null) throm new IllegalStateExeption("liste bos
   Elemon yavas = bas;
   Elemon hizli = bas;
  while(hizli != null && hizlileri != null){
      yavas = hizli.ileri;
      hizli = hizli.ileri.ileri;
     }
   return yavas.veri
  }
```

Bağlı Liste Kopyalama

1. Shallow Copy – Referansı Kopyalama

```
Elemon baskaBas =bas;
```

Bu işlem yeni düğümler oluşturmaz. İki değişken aynı ilk düğümü gösterir.



Bu nedenle düğümlerden birinin `veri` alanı değiştirilirse aynı değişiklik diğer referanstan da görülür.

Örneğin:

```java
Eleman baskaBas = bas;
baskaBas.veri = 999;
```

Burada başlangıç düğümünün verisi de 999 olur, çünkü aynı düğüm değiştirilmiştir.

8.2. Deep Copy — Bağımsız düğümler oluşturma



Veriler aynı olsa bile düğümler ayrı nesnelerdir; bir listedeki değişiklik diğerini etkilemez.

```java
BagliListe kopyala() {
    BagliListe kopya = new BagliListe();

    Eleman gecici = bas;
    Eleman kopyaSon = null;

    while (gecici != null) {
        Eleman yeni = new Eleman(gecici.veri);

        if (kopya.bas == null) {
            kopya.bas = yeni;
        } else {
            kopyaSon.ileri = yeni;
        }

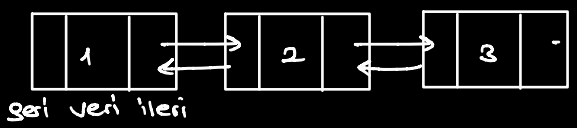
        kopyaSon = yeni;
        gecici = gecici.ileri;
    }

    return kopya;
}
```

Diğer Linked List Çeşitleri

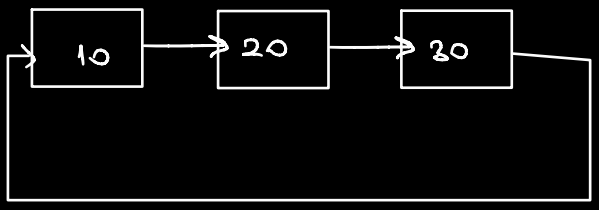
1. Doubly Linked List - Çift yönlü bağlı liste

* Her düğümde üç bölüm oluşur.




```
class CiftElemon{
    int veri;
    CiftElemon ileri;
    CiftElemon geri;
    CiftElemon(int veri){
      this.veri=veri;
      this.geri=null;
      this.ileri=null;
    }
```

2. Circular Linked List - Dairesel Bağlı Liste




```
void DairiselListe(){
    if(bas==null)return;
    Elemon gezici=bas;
    do{
      System.out.println(gezici.veri);
      gezici=gezici.ileri
    }while(gezici!=null);
}
```